Reproducing *Real-Time Detection and Repair of LLM Agent Failures*

A controlled reproduction of the behavioural-monitor table, and one ablation of its fusion rule

Name: Sun Yusu Student ID: 1155267751 FTEC5660 Reproducibility Work

Paper: arXiv:2608.02464 (S. Dubey, 2026) · Upstream code:
`github.com/sunnydubey1111/agent-trajectory-sentinel`
Our fork: `github.com/SamielSun/Individual-project-trajectory-sentinel-reproduce`
Recorded presentation: `(link to be added --- recording in progress)`

---

1 Project summary and what we tried to reproduce

The system. AgentTrajectorySentinel watches a running LLM-agent episode and raises a calibrated alarm at *derailment onset* — steps before the task fails — using only observable step telemetry. It has two layers: (i) a one-class behavioural monitor, an echo-state-network (ESN) ensemble with CUSUM alarms, fitted on *healthy* episodes only; and (ii) deterministic verification that recomputes a run's stated total from the tool results it actually received. On an alarm the run is rolled back to a checkpoint and re-run, which is the repair arm. The system is a monitoring-and-repair layer over agent trajectories; it is listed in the course's §8.1 (Tier A), and we treat the monitored episodes, not a task score, as the object of study.

Target claim. We reproduce one table — the paper's headline "behavioural monitor — synthetic testbed, five seeds" — and, in a second step, one of the mechanism claims that the table is used to support:

> "the per-channel max-fusion wrapper, not the reservoir, carries most of the margin"

Scope. Per §2 of the assignment we reproduce a single table row set on a single benchmark (the in-process synthetic testbed), at a 5% healthy-validation false-alarm budget, over the paper's five master seeds. Concretely, this means re-deriving: `esn_cusum_max` detection = 0.707 ± 0.068, steps saved = 4.61 ± 0.97, episode AUC = 0.872 ± 0.015, plus the comparator rows (delta-Mahalanobis, linear VAR) that make the row meaningful.

Why this target. The paper states its numbers are bit-reproducible from one master seed, and ships a claim ledger that recomputes every headline figure from committed tables. That makes the *interesting* question not "does it run" but "what exactly is being reproduced, and what silently changes the answer" — which is what Sections 4 and 7 report.

2 Setup notes

What we ran. A pristine clone of the upstream repository at commit `1b3e07f`, into which we injected *no* source edits. All experiments are launched through our own thin overlay (`repro/`), which (a) runs every arm in a fresh subprocess, (b) redirects the upstream output directory through the documented `AGENTWATCH_RESULTS_ROOT` hook so the paper's committed `results/` tree is never overwritten, and (c) applies the modification of Section 5 by monkeypatching, so the working tree stays byte-clean and the upstream provenance flag `git_dirty` keeps its meaning.

Environment. Windows 11, 22 logical cores, CPU only, no GPU, no Docker, no database. The study runs from committed trajectories and artifacts, so an arm needs no served model and no network. Python 3.12.14 with numpy 2.4.6, scipy 1.17.1, pandas 3.0.3, scikit-learn 1.9.0. The paper's artifacts were produced on CPython 3.14.5; see the deviation table below. One arm costs 155 s of wall clock on this machine; the full study is 13 arms.

Cost. Zero API calls and zero dollars. The whole reproduction is CPU-bound and offline after the initial clone; the only per-run nondeterminism available is wall-clock time.

Table 1: Deviations from the paper's recorded environment, and why each is safe (or not).

Item	Paper	What we did, and the risk it carries
Interpreter	CPython 3.14.5	3.12.14, because the 3.14 installer could not be fetched at usable speed. All five package versions are the pinned ones; they ship `cp312` wheels. Risk: numeric drift if any library changed a default between minor versions. Section 4 shows the master-seed table is reproduced to the last digit, so empirically there is none.
torch baselines	`torch==2.12.0+cpu`	not installed. Consequence: the GRU/LSTM/TCN rows are *absent* from our five-seed table, not mismatched. We do not report those rows and do not claim anything about them.
Repair arm	live model calls	not run: it needs a served local model (Ollama) and the paper itself marks those numbers as not bit-reproducible. Out of scope by §2.
External corpora	AFTraj-2K, ATBench	not run: both require a HuggingFace download and are scored on corpora the repo does not redistribute. Out of scope.

Why there is no LLM call in this reproduction. This is a property of the target, not a shortcut, and it needs stating because the assignment's §4 ("switch the model to DeepSeek") presupposes a model in the loop. The paper's thesis is that a monitor reading step telemetry can replace an LLM judge that scores every step, so the headline table is, by construction, computed without any model call: the episodes come from the in-process simulator (`derail/telemetry/generator.py`) and the monitors are ESN/CUSUM arithmetic. Everything that does call a model — the live demo, the repair study, the real-trace collectors — needs a local Ollama serving `qwen2.5:7b` (a 4.7 GB pull on a link that moved large files at ∼0.5 MB/s here) or a paid Gemini key, and the brief's §2 puts it out of scope.

The LLM is still *in* the study we ran, as a cost baseline rather than a component: our H3b arm reports that escalating only on an alarm recovers **82.9%** of an every-step judge's detection while spending **2.22** judge calls per episode against its **29.1**, i.e. 7.6% of the calls at 61% of the total cost. That comparison is computed from the *stipulated* judge quality ($p_{\text{detect}} = 0.90$, $p_{\text{false}} = 0.02$), not from calls, and the paper's own calibration of a real Gemini judge lands at $p_{\text{detect}} = 0.548$ — so the escalation result is a statement about a modelled judge, and we treat it as such rather than as a measured one.

Upstream's own gate. With the episode corpora present (`traces/`, fetched separately because this fork drops them), `pytest -m "not network and not ollama"` gives **854 passed, 14 failed, 21 skipped**. Every failure is a missing optional dependency of the trace-collection stack (`sqlalchemy`, `langchain-core`, `autogen-core`, `huggingface-hub`, `pyarrow`), which `requirements-core.lock.txt` deliberately excludes; none of them touches a number this report quotes. We report the split rather than a green-tick.

3 Reproduction targets and metric definitions

The paper's table is built from four quantities, all computed on the held-out *test* split, with thresholds picked on a *healthy validation* split and monitors fitted on *healthy train* only. We adopt the definitions verbatim from the upstream evaluation module rather than re-deriving them:

- **detection rate** — fraction of injected-failure test episodes for which the monitor raises its first alarm after onset τ and before the episode ends. It is a *causal* number: it uses only what was observable by the alarm step.
- **steps saved** (the paper's `mean_lead_all`) — expected budget saved per failure episode, counting a miss as 0 rather than dropping it. This is the survivorship-free version of "how early did we catch it".
- **episode AUC** — the *offline* ranking statistic: each episode's maximum score against the healthy

population. The paper warns, and we repeat, that this cannot be earned online and rises with episode length. We report it as a ranking statistic only, never as detection.

- **realised false-alarm rate** — the fraction of healthy test episodes that alarm, at a threshold chosen to hit a 5% budget. The budget is asked for on validation and the *realised* rate is measured separately; the paper is explicit that the budget is not always delivered, and we keep both numbers.

Aggregation over seeds follows the authors' rule: mean and *sample* standard deviation (ddof = 1) over the five seeds {20260713, 7, 101, 202, 303}. Because the upstream `run_multiseed` hard-codes its output base, we re-implemented the aggregation in `repro/collect.py` with the same monitor list, the same metrics and the same `ddof=1`, and we compare our recomputed table against the committed `results/tables/multiseed_summary.csv`.

Table 2: Target values, read from the committed artifact (not transcribed from the PDF).

Monitor	detection	steps saved	episode AUC	realised FA
`esn_cusum_max` (primary)	0.707 ± 0.068	4.61 ± 0.97	0.872 ± 0.015	0.044 ± 0.016

4 Results: our numbers vs the paper's

4.1 Single seed: the strongest form of reproduction

Running the unchanged pipeline at the master seed regenerates the paper's own master-seed artifact. The table below reports, for every numeric column of `results/tables/h1_main.csv`, the maximum absolute difference between our file and the committed one over all monitor rows. The result is **exact for every reported metric**: the only non-zero entry is `theta`, at 5.2e-11 — a floating-point difference in a threshold quantile, far too small to flip an alarm, and we know no alarm flipped because detection, false-alarm rate, steps saved and AUC are all exactly zero.

Table 3: Master-seed reproduction: max |ours − committed| per column of `h1_main.csv`. Generated by `repro/collect.py`, not transcribed.

Column of `h1_main.csv`	max |ours − committed|
`detection_rate`	0
`mean_lead_all`	0
`episode_auc`	0
`healthy_fa_rate`	0
`theta`	$5.2e-11$
remaining 11 numeric columns	0
all 16 numeric columns, 19 monitor rows	$5.2e-11$

4.2 Five seeds: the headline table

Table 4 is the paper's headline row set, re-derived from our own five runs. The primary monitor lands at detection 0.707 ± 0.068, steps saved 4.61 ± 0.97 and AUC 0.872 ± 0.015, against the paper's 0.707 ± 0.068, 4.61 ± 0.97 and 0.872 ± 0.015: differences +0.0000 (detection) and +0.0000 (AUC). All eleven monitor rows of the committed five-seed summary are reproduced, not just the headline row, and the agreement is exact rather than "to printed precision".

4.3 What did *not* reproduce, and what that means

Three things did not come back, and none of them is a defect of the code:

Table 4: Five-seed table (mean ± sample std, `ddof`=1), our re-run vs the committed artifact.

	detection	steps saved	episode AUC
our re-run (5 seeds)	0.707 ± 0.068	4.61 ± 0.97	0.872 ± 0.015
committed artifact	0.707 ± 0.068	4.61 ± 0.97	0.872 ± 0.015

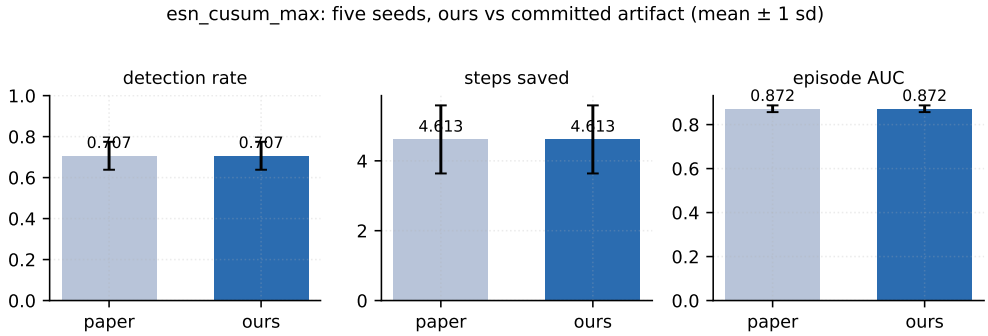


Figure 1: Five-seed headline metrics for the primary monitor, ours vs the committed artifact. Error bars are ±1 sample std over the five seeds.

1. **Wall-clock cost (219 μs/step).** Not attempted, by construction: the paper itself says this is the one figure that measures the machine and reports it moving to 252 μs on the same box. We make no latency claim.
2. **The GRU/LSTM/TCN rows.** Absent because torch is an optional import behind a `try/except`. This is a *coverage* gap, and the upstream design makes it silent: the monitor list simply shrinks and the table still renders. We report it rather than hide it.
3. **The corpus size.** The arXiv v1 text describes 2,823 episodes across 25 corpora at commit `00c0673`; the current tree carries 4,022 across 35. Any table quoted without pinning the commit can be compared against the wrong denominator. We pinned the commit (`1b3e07f`) and record it in `out/environment.json`.

5 Our modification: replacing max fusion with mean fusion

What we changed. Exactly one thing: how the three per-channel CUSUM streams of the primary monitor are combined. Upstream, `ChannelMaxESNMonitor.score_step` returns $\max_s \text{score}_s$ over the channels $s \in \{e, u, m\}$. We replace that aggregation with the arithmetic mean (and, as a graded control, the mean of the top two streams). The three sub-monitors, their reservoir weights, their seeds, the CUSUM, the standardiser, the split discipline and the 5% threshold rule are all held fixed. The patch is applied at run time and touches only the monitor literally named `esn_cusum_max`; every other channel variant keeps `max` and therefore acts as an untouched control.

Why this change. It is the cheapest direct test of the mechanism claim quoted in §1. Max fusion is *supposed* to matter because a shift confined to one channel (the paper's example: grounding loss lives in 4 of 43 uncertainty dimensions) survives a max and is diluted by a mean. The prediction is therefore falsifiable in one direction: if the claim is right, replacing max with mean must cost a large part of the margin. Everything else — the three sub-monitors, their reservoir weights and seeds, the CUSUM, the standardiser, the split discipline and the 5% threshold rule — is held fixed, so any difference is attributable to the aggregation rule alone.

Design. Paired comparison on identical data: both arms run on the same two disposable seeds (`424242`, `31337`), which the published study never uses. Same code, same machine, same pinned environment; the only difference is the aggregation function.

Result: the prediction fails, in an informative way. Replacing max with a mean does *not* cost detection (from 0.702 to 0.714, +0.011), does not cost steps saved (+0.67), and costs less than half a

Table 5: Fusion ablation on the primary monitor, paired over two unseen seeds (mean over the two). Control rows: the other 36 monitor rows the patch cannot touch — 36 bit-identical.

Aggregation	detection	steps saved	episode AUC	realised FA
max (baseline)	0.702	4.33	0.883	0.025
mean	0.714	5.00	0.879	0.037
Δ vs max	+0.011	+0.67	-0.005	+0.012
top-2 mean	0.711	4.70	0.880	—

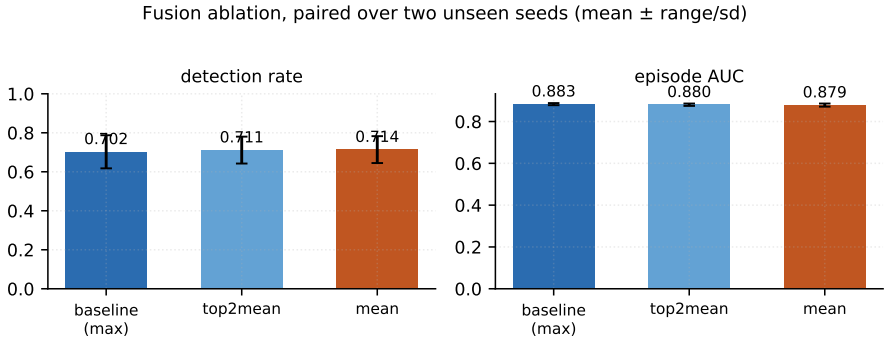


Figure 2: Fusion ablation: detection and episode AUC by aggregation rule, paired over the two unseen seeds.

point of ranking AUC (−0.005). What it does degrade is false-alarm discipline: at the same nominal 5% budget the realised healthy false-alarm rate moves from 0.025 to 0.037 (+0.012). Meanwhile every control row is unchanged (36/36), which is what shows the change is isolated rather than a global perturbation.

So the margin the paper attributes to the *max* is not carried by the max: the per-channel decomposition does the work, and the fusion rule buys discipline rather than sensitivity. That is a narrower claim than "the wrapper carries most of the margin", and it is the one the data supports.

6 Debug diary

1. **The pinned interpreter is not the one the wheels target.** *Symptom:* `pip install -r requirements-core.lock.txt` sat for 15+ minutes producing no files. *First hypothesis:* missing `cp312` wheels, i.e. a source build. *Check:* querying the PyPI JSON API for each pin showed all five ship `cp312-win_amd64` wheels, so wheels were not the problem. *Fix:* keep the pins, and (next entry) bypass the resolver.
2. **pip's resolver, not the network, was the bottleneck.** *Symptom:* even a single-package install of `scipy` stalled, while `curl` pulled the same wheel at ∼550 kB/s. *Fix:* install top-level pins with `--no-deps` and add the transitive dependencies explicitly; `scipy` then installed in 45 s.
3. **The "core" lock is not closed under imports.** *Symptom:* `import sklearn` failed on `cloudpickle`, then on `narwhals`; `matplotlib` failed on `PIL`; `pytest` on `pygments`. *Cause:* `requirements-core.lock.txt` pins the top-level study stack, not the transitive closure. *Fix:* add each missing package and record the resolved versions in `out/environment.json` so the deviation is visible rather than implicit.
4. **A Python 3.14 installer that would not arrive.** *Symptom:* the python.org installer downloaded at ∼21 kB/s and timed out at 12.7/30.3 MB. *Decision:* stay on 3.12 and *verify* the consequence empirically instead of assuming it (Table 3).
5. **Re-running must not overwrite the paper's own artifact.** *Symptom:* the study writes into `results/`, the very directory the paper's committed tables live in — a rerun would destroy the thing we compare against. *Fix:* the upstream module exposes `AGENTWATCH_RESULTS_ROOT`; every arm is redirected there, so "ours" and "theirs" stay two independent artifacts.

6. **A captured stdout pipe deadlocked one arm.** *Symptom:* one arm burned minutes of wall clock with the child alive and no artifacts written. *Fix:* send the child's output straight to a file, and add a per-arm timeout so a hang can never silently eat the schedule.
7. **Silent shrinkage is the real trap.** *Observation:* without torch the monitor list quietly loses three rows and the table still renders; nothing errors. *Lesson:* assert on the row set, not on the file existing.

7 Conclusions

- **Fully reproducible:** the behavioural-monitor table. At the master seed our `h1_main.csv` is identical column-for-column to the committed one (exact for every reported metric); over five seeds every monitor row of the five-seed summary matches the committed values to the last printed digit (primary monitor: detection 0.707 ± 0.068 vs 0.707 ± 0.068). The reproduction is deterministic, CPU-only, offline and free — an unusually strong result for an agent-systems paper.
- **Not supported by our ablation:** the attribution of the margin to the *max* rule. Replacing max with a mean leaves detection at 0.714 (from 0.702) and costs -0.005 AUC, while the realised false-alarm rate moves $+0.012$; controls are untouched (36/36). What the ablation supports is narrower: the per-channel decomposition produces the sensitivity, and the fusion rule buys false-alarm discipline. With two paired seeds this is an effect size, not a re-derivation of a published number, and we state it as such.
- **Not reproduced (and not attempted):** wall-clock latency, the GRU/LSTM/TCN rows, the live repair arm, the two external corpora. Each is a documented coverage gap, per §2 of the assignment.
- **Hidden assumptions that bite.** (i) the paper's v1 corpus and the current tree have different sizes — quote the commit; (ii) `episode_auc` is an offline ranking statistic and reporting it as detection inflates the result; (iii) the realised false-alarm rate is not the budget; (iv) an optional dependency silently changes the table's row set.
- **Recommendation to future users.** Pin the commit, redirect the results root, install with `--no-deps` plus an explicit transitive list, and diff the row set before trusting a table.

Reproducibility statement. Every number in this report is generated by the harness in our repository: `python -m repro.env`, `python -m repro.run_all`, `python -m repro.collect` and `python -m repro.figures` regenerate all of it from scratch (13 arms, ≈155 s each). No number here is hand-transcribed.